

Silent-Radix Cryptography:

Exploiting the Base Ambiguity of Positional Notation as a Cryptographic Primitive

Author: QNFO Research Collective | **Date:** 2026-06-29 **License:** QNFO Unified License Agreement (QNFO-ULA): <https://legal.qnfo.org/>

Abstract

The observation that a positional numeral cannot internally specify its own base — the “silent radix” problem identified in §1.1 of the Ultrametric Foundation — is not merely a philosophical curiosity. It is a cryptographic primitive. We formalize this as **Silent Radix Encryption (SRE)**, a key encapsulation mechanism (KEM) where the shared secret is derived from the base- b interpretation of publicly transmitted decimal digits. Only a party who knows the secret base b can recover the correct key. Eve, seeing the same decimal digit string N , must solve a variant of the integer knapsack problem where the weights (b^i) are unknown: given $N = \sum d_i \cdot 10^i$, find b such that $K = \sum d_i \cdot b^i$ decodes the ciphertext correctly. For $b \sim 2^{128}$, brute force is infeasible.

1. Mathematical Foundation

1.1 The Silent Radix

In any integer base $b \geq 2$, the digit string “10” denotes the number b . This holds universally: binary “10” = two, decimal “10” = ten, sexagesimal “10” = sixty. Therefore every base, when named using its own numeral system, calls itself “base-10.” [established - §1.1 of the Ultrametric Foundation]

The efficiency of positional notation is purchased at the permanent cost of latent ambiguity. A numeral string cannot internally specify its own interpretation rule. The radix is always a **silent parameter**, supplied by an external meta-language — a human convention, a hardware specification, a cultural default.

This ambiguity is the cryptographic resource exploited by SRE.

1.2 The Native Ultrametric

The b -adic valuation $d(x, y) = b^{-v_b(x-y)}$ satisfies the strong triangle inequality [established - §2.1, Ultrametric Foundation]:

$$d(x, z) \leq \max\{d(x, y), d(y, z)\}$$

This ultrametric is the native geometry of positional notation. The digit string is a path through a rooted tree; the distance between two numbers is the depth of their lowest common ancestor — exactly the count of trailing matching digits.

The cryptographic significance: two numbers that appear “close” in the Archimedean sense can be “far” in the ultrametric tree (differing in many trailing digits), and vice versa. The encoding of a

key depends on which metric the observer uses — and Eve lacks the base needed to compute the correct metric.

2. Silent Radix Encryption (SRE)

2.1 Scheme Overview

SRE is a **non-interactive key encapsulation mechanism** (KEM) built on the silent-radix ambiguity. No prior communication between Alice and Bob is needed beyond pre-sharing the secret base b .

Alice (encapsulation):

1. Generate k random decimal digits $D = [d_0, \dots, d_{k-1}]$, each $d_i \in \{0, 1, \dots, 9\}$.
2. Compute the shared secret using the **base- b interpretation** of D :

$$K = \sum_{i=0}^{k-1} d_i \cdot b^i$$

3. Derive a symmetric key from K via a key derivation function (SHA-256):

$$k_{\text{sym}} = \text{SHA-256}(K)$$

4. Encrypt the message m with k_{sym} using counter-mode XOR:

$$c = m \oplus \text{PRNG}(k_{\text{sym}})$$

5. Transmit the **public envelope** — the decimal interpretation of D :

$$N = \sum_{i=0}^{k-1} d_i \cdot 10^i$$

along with the ciphertext c .

Bob (decapsulation):

1. Extract the decimal digits from N :

$$d_i = (N \div 10^i) \bmod 10, \quad i = 0, \dots, k-1$$

2. Compute $K' = \sum d_i \cdot b^i$ using the **same digits, different base**.
3. Derive $k'_{\text{sym}} = \text{SHA-256}(K')$.
4. Decrypt: $m = c \oplus \text{PRNG}(k'_{\text{sym}})$.

Since the digits d_i are identical in both interpretations, $K = K'$ and $k_{\text{sym}} = k'_{\text{sym}}$. The scheme is **correct**.

2.2 Why It Works

The ciphertext has **two layers**:

Layer	Content	Visible to Eve?
Envelope	$N = \sum d_i \cdot 10^i$ (decimal number)	Yes — this is the public communication
Payload	$c = m \oplus \text{PRNG}(k_{\text{sym}})$ (encrypted message)	Yes, but encrypted

Eve sees N — a decimal number. She can extract its decimal digits d_i . But those digits, interpreted in a *wrong* base $b' \neq b$, produce a **different** key:

$$K_{b'} = \sum d_i \cdot (b')^i \neq \sum d_i \cdot b^i = K$$

For each candidate b' , Eve computes a candidate key and attempts decryption. The probability that a random b' produces a plausible plaintext is negligible for messages of substantial length.

2.3 Prototype Implementation

```
# Core SREv2: 512 decimal digits, 64+ bit secret base
b = keygen(64)           # b ~ 2^64
N, k, ct = encapsulate(msg, b)
pt = decapsulate(N, k, ct, b)
assert msg == pt        # Correctness verified

# Eve tries wrong bases - all produce garbage
for wrong_b in [10, 16, 100, 256]:
    assert decapsulate(N, k, ct, wrong_b) != msg
```

The prototype demonstrates: - **Correctness**: the correct base b recovers the message exactly. - **Ambiguity**: four wrong bases all produce cryptographically random output. - **Key sizes**: 512 decimal digits for $\lambda = 64$, scaling to 4096 digits for $\lambda = 128$.

3. Security Analysis

3.1 Reduction to Integer Knapsack

Eve's problem: given $N = \sum d_i \cdot 10^i$ (known), find b such that $K = \sum d_i \cdot b^i$ produces a decryption key that yields a recognizable plaintext.

This is equivalent to: given the decimal expansion of N , find a base b such that when the digits are reinterpreted in base b , the resulting integer K serves as a symmetric key. The digits d_i are fixed by N ; only b is free.

Eve faces an **unknown-weight knapsack problem**: compute $K(b) = \sum d_i \cdot b^i$ for all $b \in [10, 2^\lambda]$, derive $k_{\text{sym}}(b) = \text{SHA-256}(K(b))$, and attempt decryption. No known shortcut exists beyond enumeration.

3.2 Brute-Force Complexity

For a secret base $b \sim 2^\lambda$:

Security parameter λ	Candidate bases	Brute-force cost	Status
64	$\sim 2^{64}$	Feasible for large organizations	Acceptable for prototype
128	$\sim 2^{128}$	Infeasible for classical computers	Post-quantum resistant
256	$\sim 2^{256}$	Infeasible for any foreseeable computer	Excessive margin

The brute-force attack requires: for each candidate b , compute $K = \sum d_i \cdot b^i$ (an $O(k \log b)$ arithmetic operation), derive k_{sym} , decrypt c , and check for valid plaintext. With $k = 4096$ digits for $\lambda = 128$, each attempt costs ~ 4096 multi-precision multiplications. At $\sim 10^9$ ops/sec, this is $\sim 4 \mu\text{s}$ per attempt, or $\sim 2^{64}$ seconds to exhaust a $\lambda = 128$ key space — well beyond any practical horizon.

3.3 Known-Plaintext Attacks

If Eve knows a plaintext-ciphertext pair (m, c) , she can compute $k_{\text{sym}} = m \oplus c$ and work backward to find K . However, K is one of infinitely many pre-images of $\text{SHA-256}(K) = k_{\text{sym}}$. Hash pre-image resistance (128 bits for SHA-256 against quantum adversaries in Grover’s algorithm) protects against this attack.

3.4 Lattice Reduction

The problem of finding b such that $K(b) = \sum d_i \cdot b^i$ matches a target can be cast as finding integer solutions to polynomial equations. For degree k , this is related to Coppersmith’s method for finding small roots of polynomials modulo integers. However, the polynomial has k terms and degree $k - 1$, which exceeds the range where Coppersmith’s method is efficient for $k > 64$.

4. Comparison with Existing Schemes

Scheme	Type	Hard Problem	Key Size	Post-Quantum?
SREv2	KEM	Unknown-weight knapsack	512–4096 decimal digits	
RSA-OAEP	KEM	Integer factorization	2048–4096 bits	
ECDH	KEM	Elliptic curve discrete log	256–521 bits	
Kyber (NIST PQC)	KEM	Module-LWE (lattice)	800–1568 bytes	
Classic McEliece	KEM	Goppa code decoding	261,120–1,357,824 bytes	

Advantages of SRE: - **No structured mathematics:** unlike lattice or code-based schemes, SRE requires only elementary arithmetic (multiplication, addition, SHA-256). - **Transparent security:** the adversary’s problem is explicit — enumerate bases, compute keys, test decryption. No algebraic structure to exploit. - **Connection to mathematical foundations:** the scheme is a direct application of the silent-radix observation, making it pedagogically transparent and theoretically grounded.

Disadvantages of SRE: - **Large envelope size:** for $\lambda = 128$, the envelope N requires ~ 4096 decimal digits ($\sim 13,600$ bits), larger than Kyber (1,568 bytes = 12,544 bits) but smaller than Classic McEliece (1.3 MB). - **No formal reduction:** the security of SRE is not yet reduced to a standard cryptographic assumption. The “unknown-weight knapsack” problem is novel and lacks cryptanalytic study. - **Linearly homomorphic:** addition of envelopes produces a new envelope whose digits are the sum of the original digits (with carries). This may enable attacks not yet discovered.

5. Extensions and Variants

5.1 Silent-Radix Signatures

A digital signature scheme can be constructed from the same primitive: - The signer publishes $N = \sum d_i \cdot 10^i$. - The signature is a proof of knowledge of b such that $H(N \parallel m) = \text{commit}(b)$, where H is a hash function and $\text{commit}(b) = \text{SHA-256}(\sum d_i \cdot b^i)$. - Verification: compute $\text{commit}(b')$ from the prover’s claimed b' and check against the published commitment. Without knowing b , forging a signature requires finding a pre-image of H .

5.2 Hierarchical Silent Radix

Multiple secret bases can be arranged in a **tree structure**, mirroring the ultrametric tree of positional notation: - Root base b_0 encrypts b_1 . - Each node uses its parent’s base for key encapsulation. - Hierarchical key derivation: $K_i = \text{SHA-256}(K_{i-1} \parallel b_i)$.

This enables group communication schemes where subtrees share nested keys, analogous to hierarchical identity-based encryption but using silent-radix as the encapsulation primitive.

5.3 Multi-Base Key Encapsulation

Instead of a single secret base b , use a vector of bases $\mathbf{b} = (b_0, b_1, \dots, b_{k-1})$:

$$K = \sum_{i=0}^{k-1} d_i \cdot b_i$$

This is a **subset-product** problem (generalizing subset sum), where the weights are independent random bases. Security is conjectured higher than single-base SRE because the adversary must find the entire vector $\mathbf{b}$, not just a single value.

6. Limitations and Open Problems

1. **Security proof.** The silent-radix KEM lacks a reduction to a standard cryptographic assumption. Formalizing the “unknown-weight knapsack” problem and proving its hardness is an open research question.
 2. **Chosen-ciphertext attacks.** The current scheme is only CPA-secure. A CCA-secure variant (using a Fujisaki-Okamoto transform or similar) should be developed.
 3. **Digit distribution.** The digits of N are uniformly random (0–9), which may be detectable by traffic analysis. Padding or re-randomization can obfuscate this.
 4. **Large envelope size.** For practical deployment, the envelope size must be reduced. Potential approaches include:
 - Compressing N using the hereditary binary encoding (Tarau 2013).
 - Using a smaller digit set (base-2 or base-4) with more digits.
 - Block-based encoding with shorter per-block envelopes.
 5. **Quantum attacks.** Grover’s algorithm provides a quadratic speedup for brute-force key search (from 2^{128} to 2^{64}). The scheme remains secure at $\lambda = 256$ against quantum adversaries. Whether the unknown-weight knapsack problem has sub-exponential quantum algorithms (beyond Grover) is unknown.
-

7. Connection to the Ultrametric Foundation Thesis

SRE is not merely a cryptographic application. It is a concrete demonstration that the **silent radix is a resource**. The same ambiguity that causes programming bugs (confusing decimal 10 with binary 10) and spacecraft losses (the Mars Climate Orbiter’s unit mismatch) can be harnessed for cryptographic protection.

The ultrametric tree of positional notation provides the geometry: - The digit string D is a **path in the tree**. - The interpretation depends on the **base** — the branching factor. - Eve traverses the tree with the wrong branching factor and arrives at the wrong leaf. - Bob, knowing b , traverses the correct tree and reaches the correct key.

This reframes cryptography not as the hardness of a number-theoretic problem but as the **irreducible ambiguity of representation** — a property native to all symbolic systems, understood in positional notation since the Babylonians but only now exploited as a security primitive.

8. Conclusion

Silent Radix Encryption demonstrates that the core insight of the Ultrametric Foundation — positional notation is natively an ultrametric tree with a silent base — has practical cryptographic consequences. The base ambiguity that has been a source of error throughout computing history can be inverted into a security primitive. While the scheme requires further cryptanalytic study and optimization before production deployment, it establishes a new connection between the philosophy of mathematical representation and the practice of information security.

This chapter extends the Ultrametric Foundation thesis (§1.1, §2.1) with a concrete cryptographic application. Prototype implementation available at `_silent_radix.py`.